



THE UNIVERSITY OF DA NANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
IT FACULTY

DIGITAL SIGNAL PROCESSING SPEECH AND SILENCE SEGMENTATION

Instructor: **Dr. Ninh Khánh Duy**

Class: **24.Nh12**

Group: **25**

Students:

- **Phan Thanh Duy** – 102240130 – 24T_DT3
- **Nguyễn Hoàng Hiếu** – 102240135 – 24T_DT3
- **Huỳnh Hiếu Nghĩa** – 102240152 – 24T_DT3

TABLE OF CONTENTS



01

Histogram
Algorithm

02

Simple Statistics
Algorithm

03

Dual-Threshold
Endpoint Detection

04

Comparison Table



The background features decorative geometric patterns in the corners, consisting of thin blue lines, dots, and circles. In the top-left, there are several parallel lines and a small cluster of dots. In the top-right, a circle with a dot inside is connected to a line. In the bottom-left, there are more parallel lines and a small cluster of dots. In the bottom-right, there are several parallel lines and a small cluster of dots.

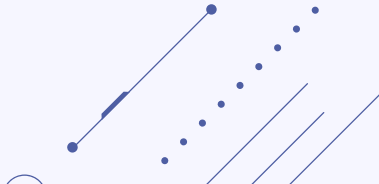
01

HISTOGRAM ALGORITHM

Presented by Phan Thanh Duy

01 HISTOGRAM: IDEA

Start



01

HISTOGRAM: IDEA

Start

Audio Signals
(.wav and .lab files)

TinHieuHuanLuyen

- phone_F1.lab
- phone_F1.wav
- phone_M1.lab
- phone_M1.wav
- README.txt
- studio_F1.lab
- studio_F1.wav
- studio_M1.lab
- studio_M1.wav

Read files from
TinHieuHuanLuyen directory.

```
def read_wav(path):  
    with warnings.catch_warnings():  
        warnings.simplefilter('ignore')  
        fs, x = wavfile.read(path)  
  
    if x.ndim > 1:  
        x = x.mean(axis=1)  
  
    if np.issubdtype(x.dtype, np.integer):  
        x = x / float(np.iinfo(x.dtype).max + 1)  
  
    return x.astype(np.float64), fs  
  
def read_lab(Lab_path, speech_labels=SPEECH_LABELS):  
    segs = []  
  
    # No annotation file -> no ground truth  
    if not os.path.exists(Lab_path):  
        return segs  
  
    with open(Lab_path, 'r') as f:  
        for line in f:  
            p = line.strip().split()  
  
            if len(p) >= 3 and p[2].lower() in speech_labels:  
                s, e = float(p[0]), float(p[1])  
                if segs and abs(segs[-1][1] - s) < 1e-9:  
                    segs[-1][1] = e  
                else:  
                    segs.append([s, e])  
  
    return segs
```

01

HISTOGRAM: IDEA

Start

Audio Signals
(.wav and .lab files)

Break the signals into
25ms frames, 10ms shift.

```
def create_frames(signal, fs, frame_ms=FRAME_MS, shift_ms=SHIFT_MS):  
    # Frame length and shift in samples  
    L = int(round(frame_ms * fs / 1000))  
    S = int(round(shift_ms * fs / 1000))  
  
    # Number of complete frames  
    n = (len(signal) - L) // S + 1  
  
    # Index matrix: row i = [i*S, i*S + 1, ..., i*S + L - 1]  
    idx = np.arange(L)[None, :] + S * np.arange(n)[:, None]  
    return signal[idx]
```

FRAME_MS = 25 (ms)

SHIFT_MS = 10 (ms)

```
# Load and frame the signal  
signal, fs = read_wav(path)  
frames = create_frames(signal, fs)
```

01

HISTOGRAM: IDEA

Extract STE and Spectral Centroid (SC) features.

Start

Audio Signals
(.wav and .lab files)

Break the signals into
25ms frames, 10ms shift.

```
def compute_ste(frames):
    # Mean of squared samples in each frame
    ste = np.mean(frames ** 2, axis=1)

    # Normalize by the largest value
    m = np.max(ste)
    return ste / m if m > 0 else ste
```

For i-th frame:

$$STE = \frac{1}{N} \sum_{n=1}^N x_i^2(n)$$

```
def compute_spectral_centroid(frames):
    # Magnitude spectrum and bin indices 1 .. N/2
    X = dft_magnitude_half(frames)
    k = np.arange(1, X.shape[1] + 1)[None, :]

    # Weighted mean of bin index (guard against
    # division by zero)
    s = np.sum(X, axis=1)
    sc = np.where(s > 0, np.sum(k * X, axis=1) / np.
        where(s > 0, s, 1), 0.0)

    # Normalize by the largest value
    m = np.max(sc)
    return sc / m if m > 0 else sc
```

For i-th frame:

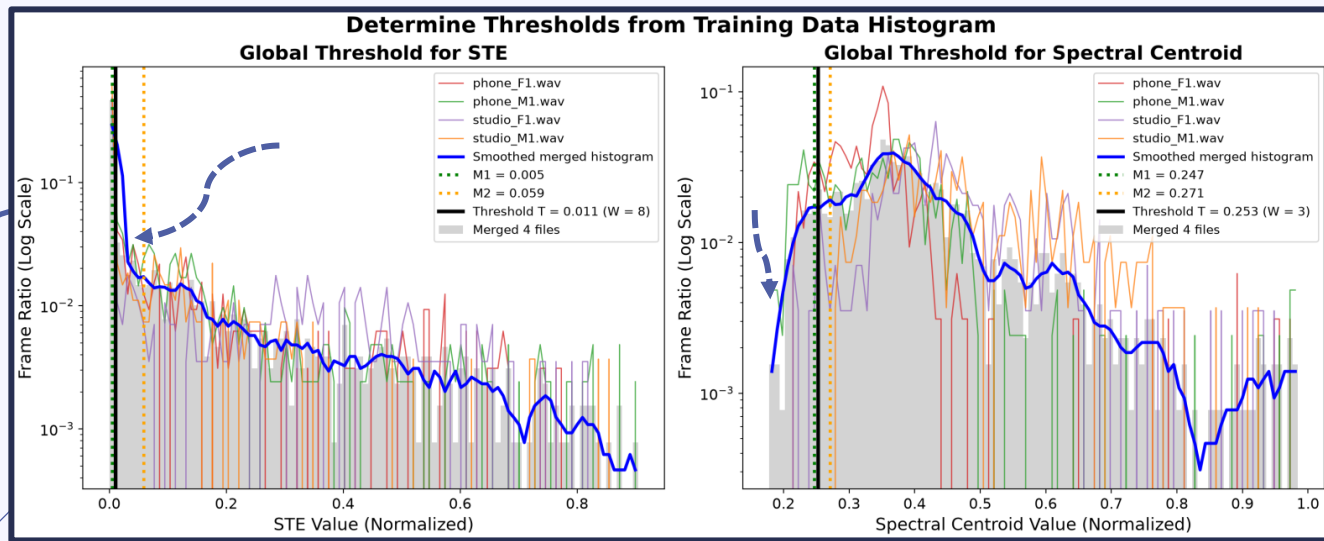
$$SC = \frac{\sum_{k=1}^N (k+1) X_i(k)}{\sum_{k=1}^N X_i(k)}$$

01

HISTOGRAM: IDEA

Merge data to compute a combined histogram.

Apply a smoothing filter to the histogram.



Start

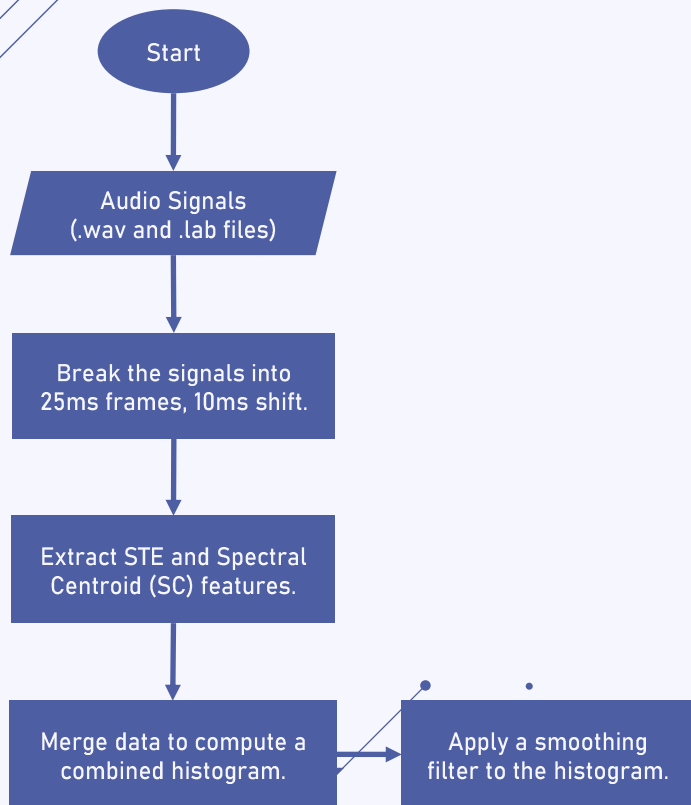
Audio Signals
(.wav and .lab files)

Break the signals into
25ms frames, 10ms shift.

Extract STE and Spectral
Centroid (SC) features.

01

HISTOGRAM: IDEA



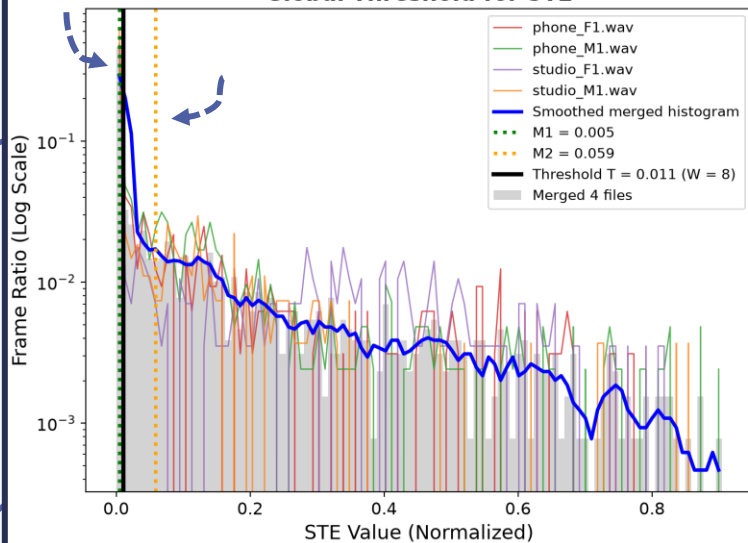
01

HISTOGRAM: IDEA

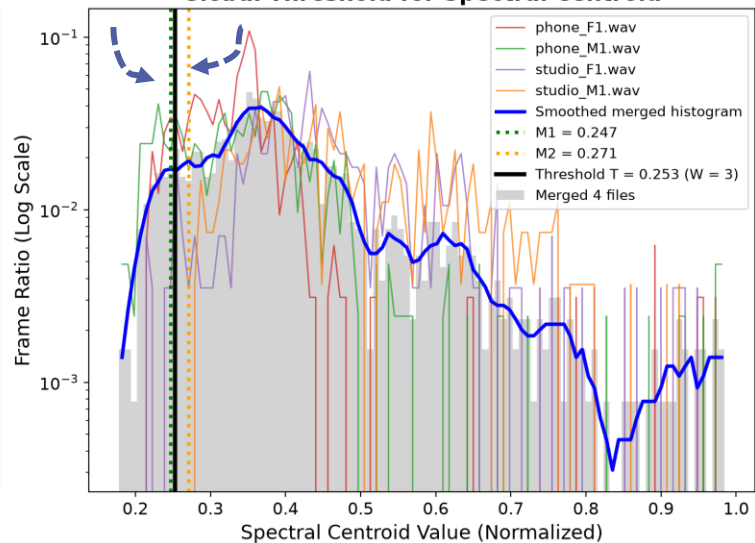
Find two local maxima peaks (M1 and M2).

Determine Thresholds from Training Data Histogram

Global Threshold for STE



Global Threshold for Spectral Centroid



Apply a smoothing filter to the histogram.

01

HISTOGRAM: IDEA

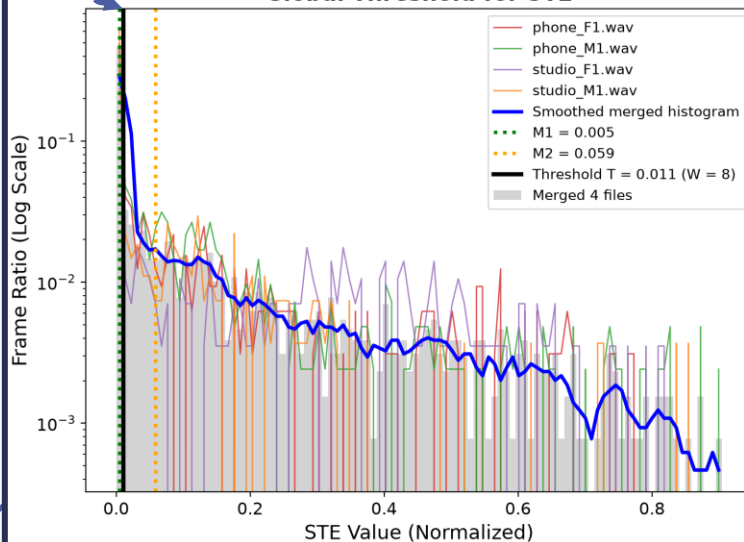
Calculate common thresholds T1 (STE) and T2 (SC).

$$T = \frac{WM_1 + M_2}{W + 1}$$

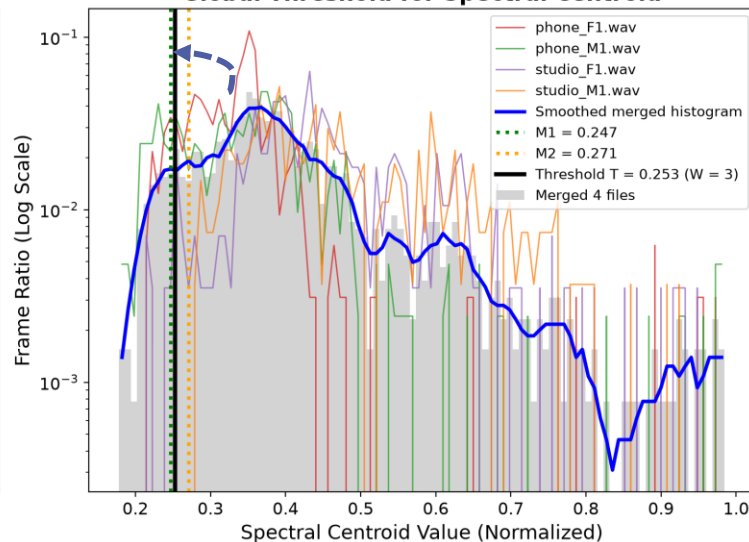
(W is user-defined)

Determine Thresholds from Training Data Histogram

Global Threshold for STE



Global Threshold for Spectral Centroid

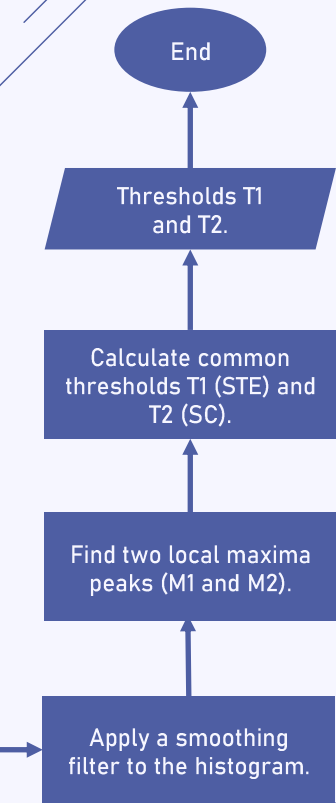


Find two local maxima peaks (M1 and M2).

Apply a smoothing filter to the histogram.

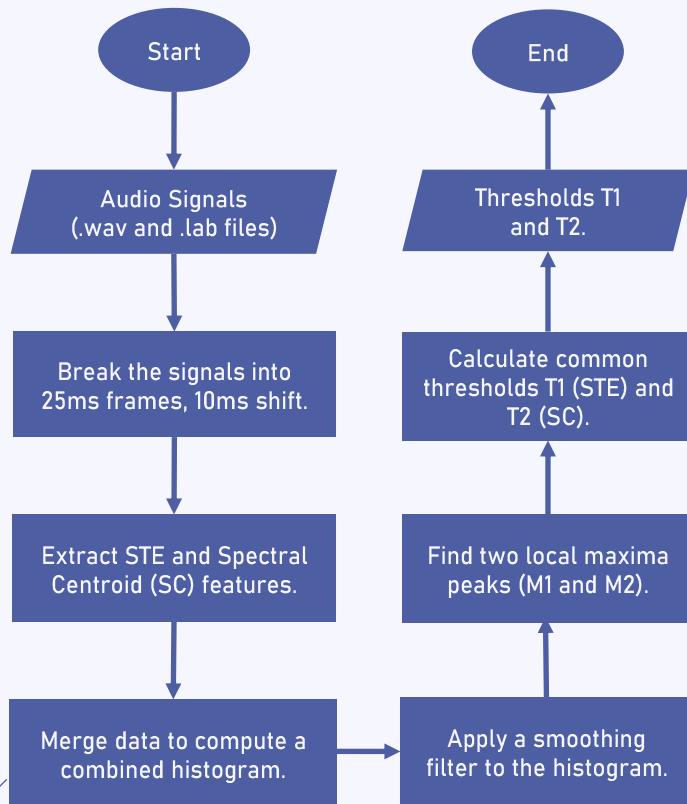
01

HISTOGRAM: IDEA



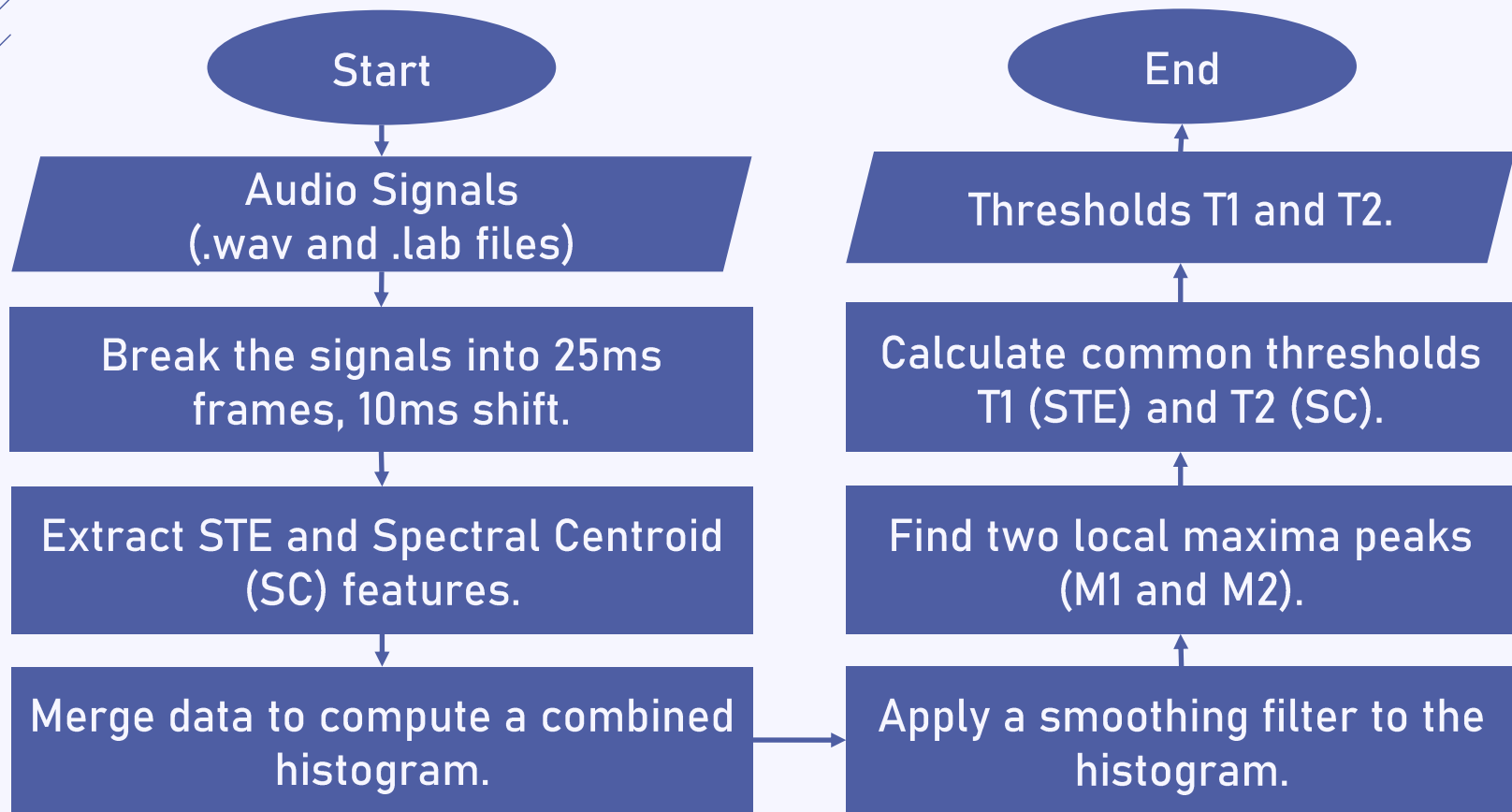
01

HISTOGRAM: IDEA



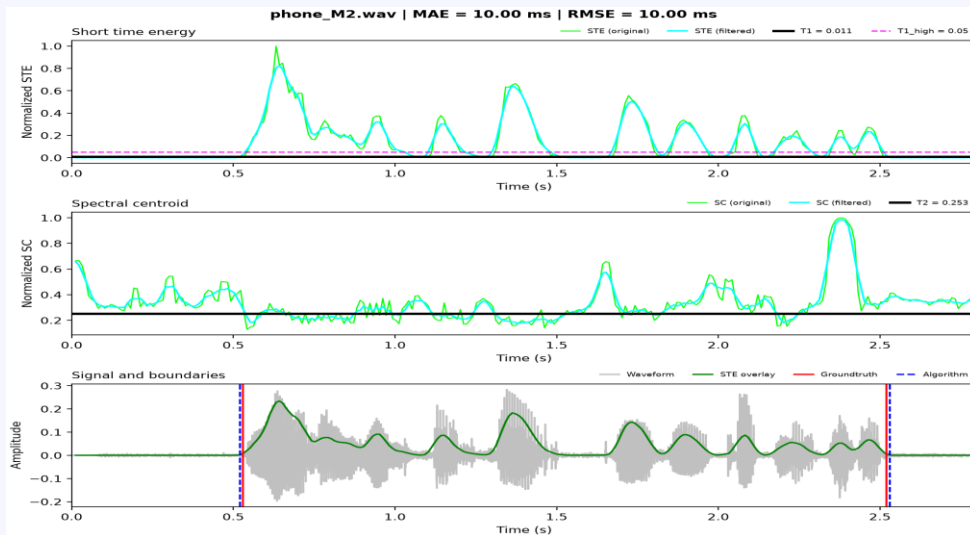
01

HISTOGRAM: IDEA



01

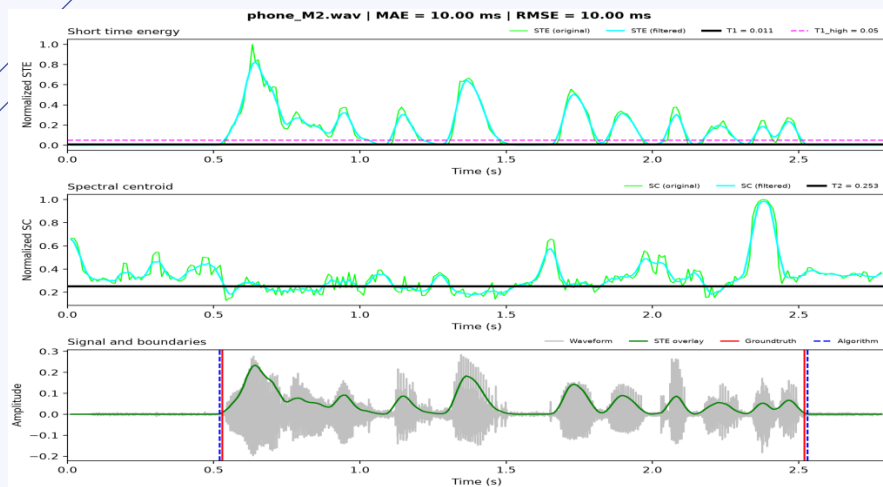
HISTOGRAM: IDEA



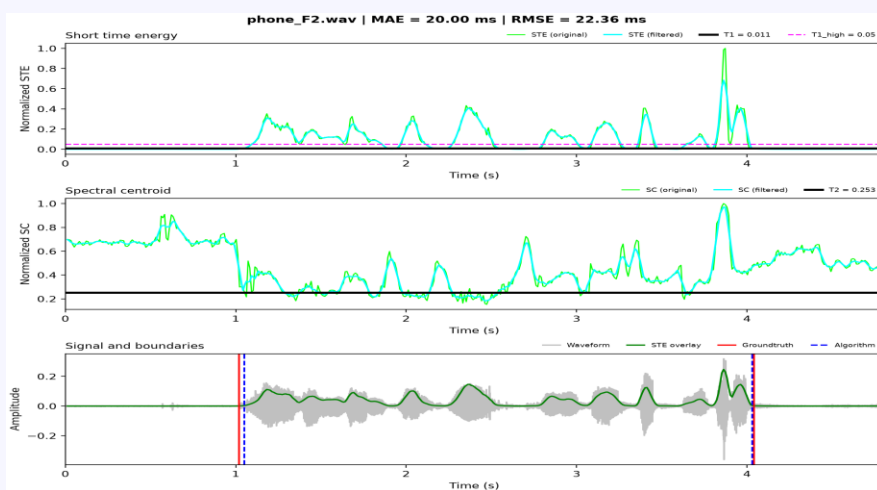
Repeat the algorithm for each Testing signal. After computing the histograms and filtering, for each frame:

- **(If STE > T1 & SC > T2) OR (STE > T1_high) => Speech (Including silence < 200ms).**
- **Otherwise => Silence.**

01 HISTOGRAM: RESULTS

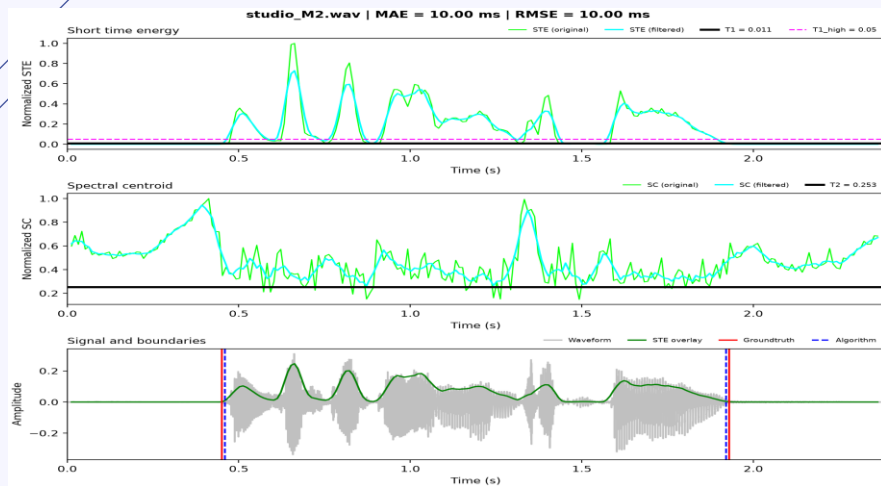


- MAE **10ms** | RMSE **10ms**
- **Accuracy:** Highly accurate, very few errors.
 - **Reason:** STE effectively captures low-frequency male voice energy. The 10ms error is just frame shift.



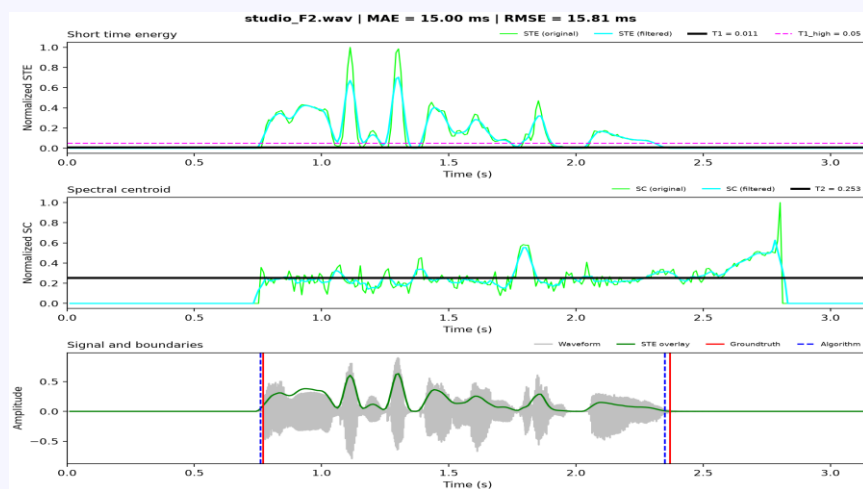
- MAE **20ms** | RMSE **22.36ms**
- **Accuracy:** Slight boundary deviation at speech edges.
 - **Reason:** Female voices contain unvoiced sounds (low STE) at word edges, causing early or late cuts.

01 HISTOGRAM: RESULTS



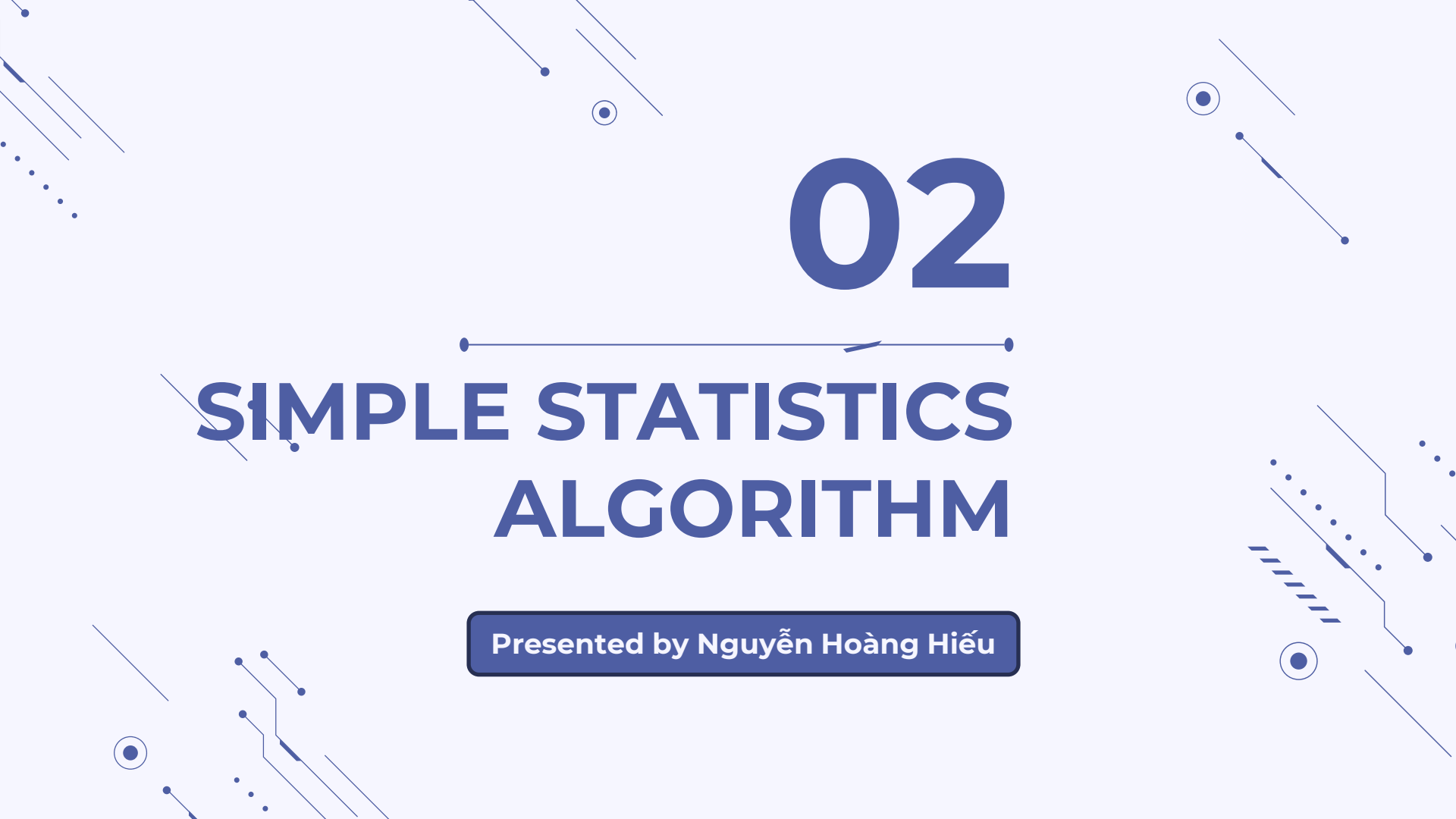
MAE **10ms** | RMSE **10ms**

- **Accuracy:** Highly accurate, very few errors.
- **Reason:** STE effectively captures low-frequency male voice energy. The 10ms error is just frame shift.



MAE **15ms** | RMSE **15.81ms**

- **Accuracy:** Slight boundary deviation at speech edges.
- **Reason:** Female voices contain unvoiced sounds (low STE) at word edges, causing early or late cuts.

The slide features decorative geometric patterns in the corners, consisting of thin blue lines, dots, and circles. In the top-left, there are several parallel lines and a small cluster of dots. In the top-right, a circle with a dot inside is connected to a line. In the bottom-left, there are more parallel lines and a small cluster of dots. In the bottom-right, there is a more complex pattern with multiple lines, dots, and a circle with a dot inside.

02

A horizontal line with a small arrow pointing to the right in the center.

SIMPLE STATISTICS ALGORITHM

Presented by Nguyễn Hoàng Hiếu

02 SIMPLE STATISTICS: IDEA

Start

02 SIMPLE STATISTICS: IDEA

Start

Audio Signals
(.wav and .lab files)

▼ TinHieuHuanLuyen

- phone_F1.lab
- phone_F1.wav
- phone_M1.lab
- phone_M1.wav
- README.txt
- studio_F1.lab
- studio_F1.wav
- studio_M1.lab
- studio_M1.wav

Read files from
TinhieuHuanLuyen directory.

```
def read_audio(path):
    # Open the file and read the header information and the raw bytes
    with wave.open(path, 'rb') as wav_file:
        fs = wav_file.getframerate()          # samples per second
        n_samples = wav_file.getnframes()     # total number of samples
        raw = wav_file.readframes(n_samples)  # raw bytes of all samples

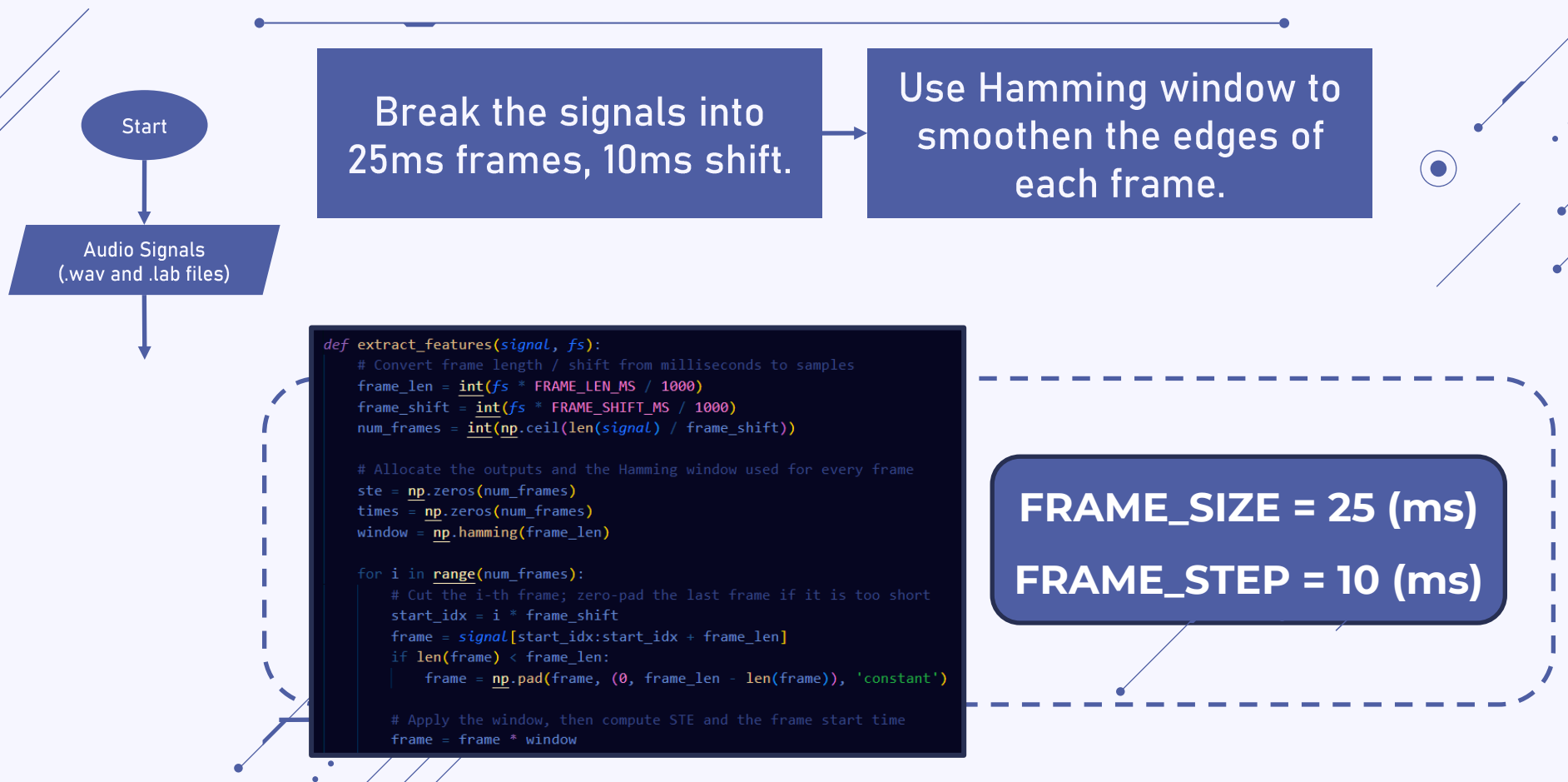
    # Convert the bytes to 16-bit integers (standard .wav format)
    signal = np.frombuffer(raw, dtype=np.int16)

    # Normalize the amplitude to [-1, 1]; an all-zero file is kept unchanged
    max_amp = np.max(np.abs(signal))
    signal = signal / max_amp if max_amp > 0 else signal.astype(float)

    # Build the time axis in seconds
    t_sig = np.linspace(0, n_samples / fs, num=n_samples)
    return signal, fs, t_sig

def read_lab_file(Lab_path):
    intervals = []
    with open(Lab_path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            # Keep only "start end label" lines and skip the F0 statistics lines
            if len(parts) == 3 and parts[0] not in ['F0mean', 'F0std']:
                intervals.append((float(parts[0]), float(parts[1]), parts[2]))
    return intervals
```

02 SIMPLE STATISTICS: IDEA



02 SIMPLE STATISTICS: IDEA

Calculate STE value of each frame.

Start

Audio Signals
(.wav and .lab files)

Break the signals into
25ms frames, 10ms shift.

Use Hamming window to
smoothen the edges of
each frame.

```
ste[i] = np.sum(frame ** 2)
times[i] = start_idx / fs

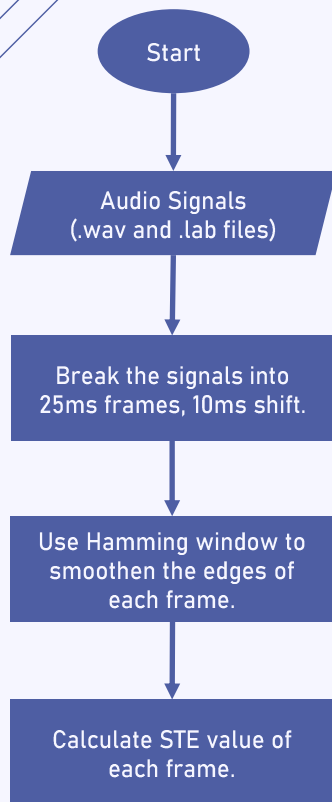
# Normalize by the maximum so that STE is comparable between files
if np.max(ste) > 0:
    ste = ste / np.max(ste)
return ste, times
```

For w-th frame:

$$STE = \sum_{n=1}^N x_w^2(n)$$

02 SIMPLE STATISTICS: IDEA

Determine the speech threshold using normal distribution statistics

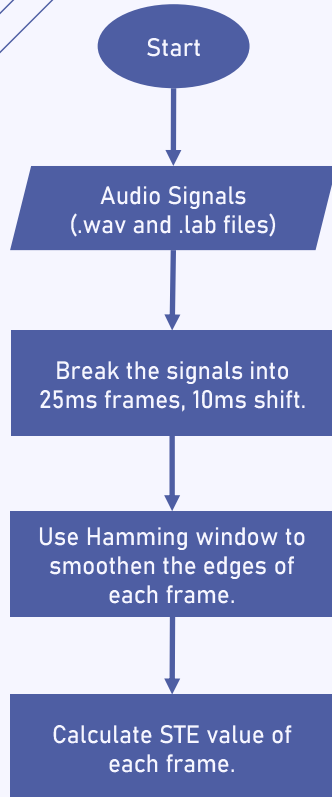


Let x be the STE threshold. Intersection of probabilities: $P_{\text{sil}}(x) = P_{\text{sp}}(x)$.

Assume STE follows a Gaussian distribution. We got :

$$\frac{1}{\sigma_{\text{sil}}\sqrt{2\pi}} e^{-\frac{(x-\mu_{\text{sil}})^2}{2\sigma_{\text{sil}}^2}} = \frac{1}{\sigma_{\text{sp}}\sqrt{2\pi}} e^{-\frac{(x-\mu_{\text{sp}})^2}{2\sigma_{\text{sp}}^2}}$$

02 SIMPLE STATISTICS: IDEA



Taking the natural logarithm of both sides, we get the quadratic equations: $ax^2 + bx + c = 0$, where:

$$a = \frac{1}{2\sigma_{\text{sil}}^2} - \frac{1}{2\sigma_{\text{sp}}^2}$$

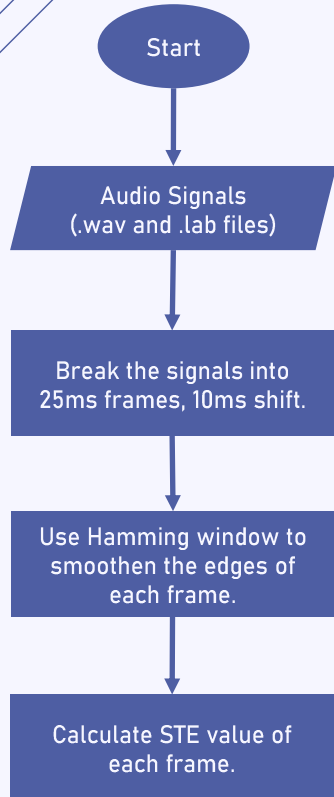
$$b = \frac{\mu_{\text{sp}}}{\sigma_{\text{sp}}^2} - \frac{\mu_{\text{sil}}}{\sigma_{\text{sil}}^2}$$

$$c = \frac{\mu_{\text{sil}}^2}{2\sigma_{\text{sil}}^2} - \frac{\mu_{\text{sp}}^2}{2\sigma_{\text{sp}}^2} - \ln\left(\frac{\sigma_{\text{sp}}}{\sigma_{\text{sil}}}\right)$$

- μ_{sp} is the speech mean (meanSp)
- μ_{sil} is the silence mean (meanSI)
- σ_{sp} is the speech standard deviation (stdSp)
- σ_{sil} is the silence standard deviation (stdSI)

02 SIMPLE STATISTICS: IDEA

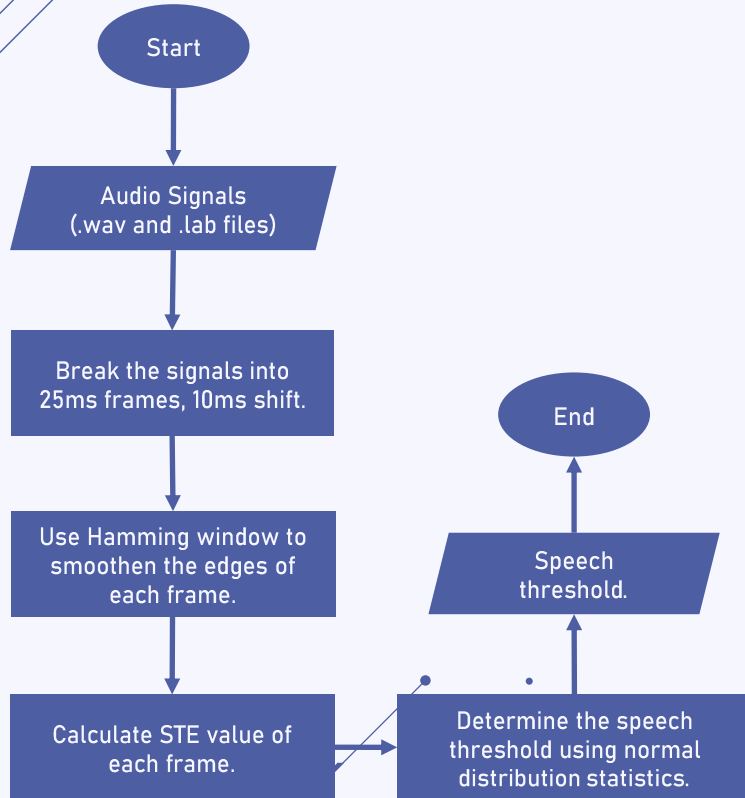
Determine the speech threshold using normal distribution statistics



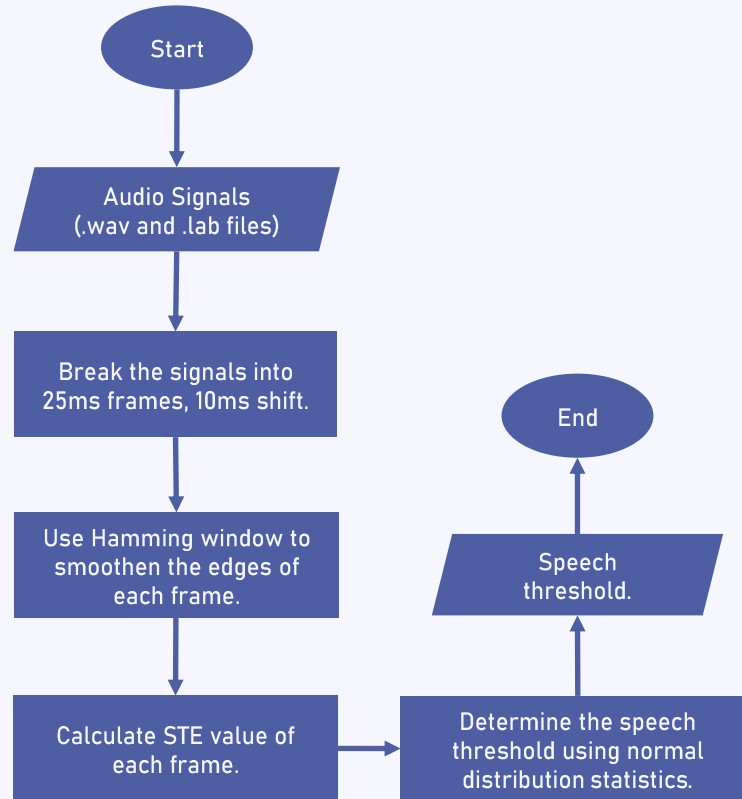
$$\Rightarrow \mu_{\text{sp}} = 0.1697, \mu_{\text{sil}} = 0.0014, \sigma_{\text{sp}} = 0.2071, \sigma_{\text{sil}} = 0.0235$$

Substituting these values into the formula above, we calculate the optimal STE threshold to be:
x = 0.0522.

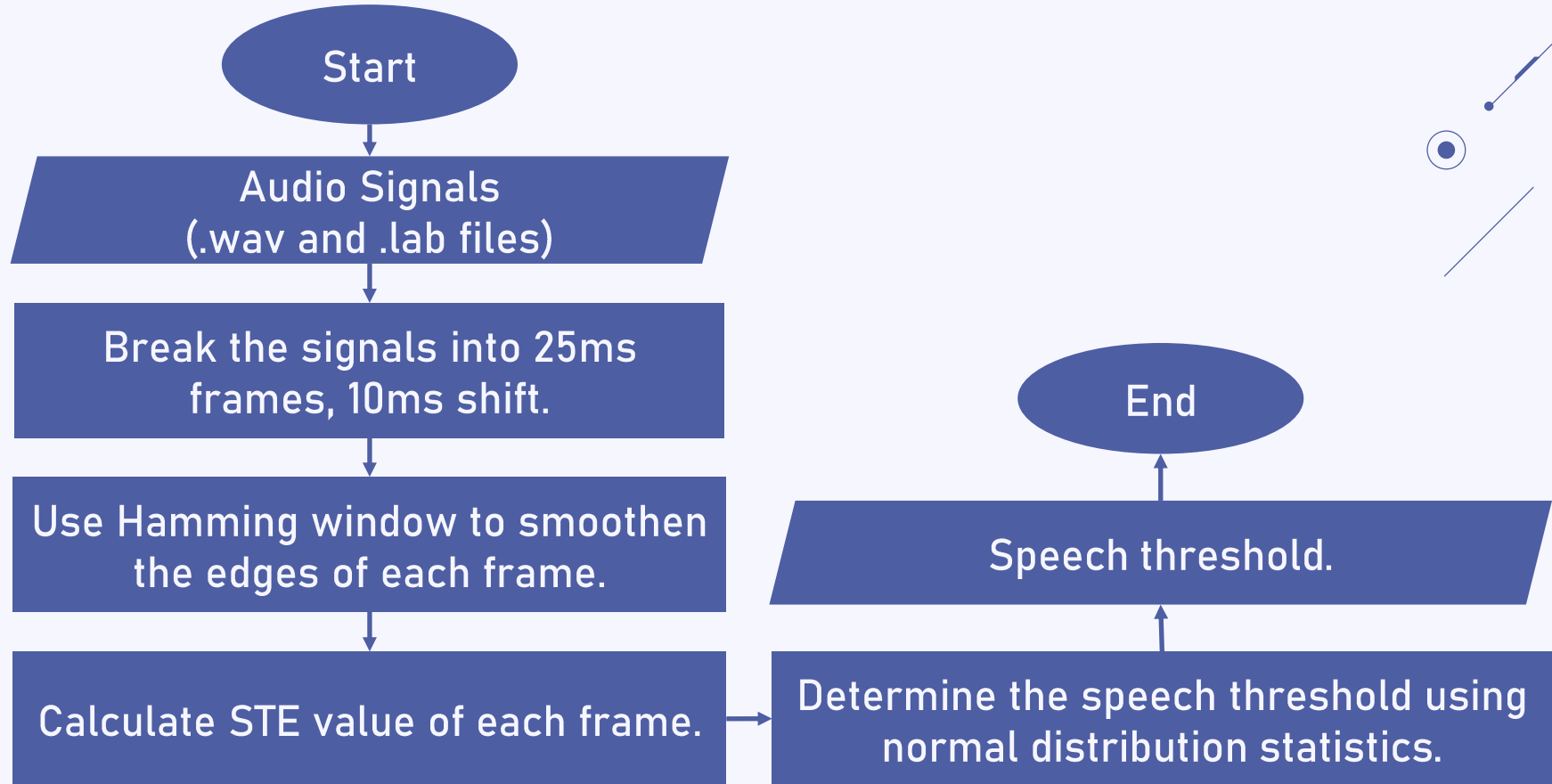
02 SIMPLE STATISTICS: IDEA



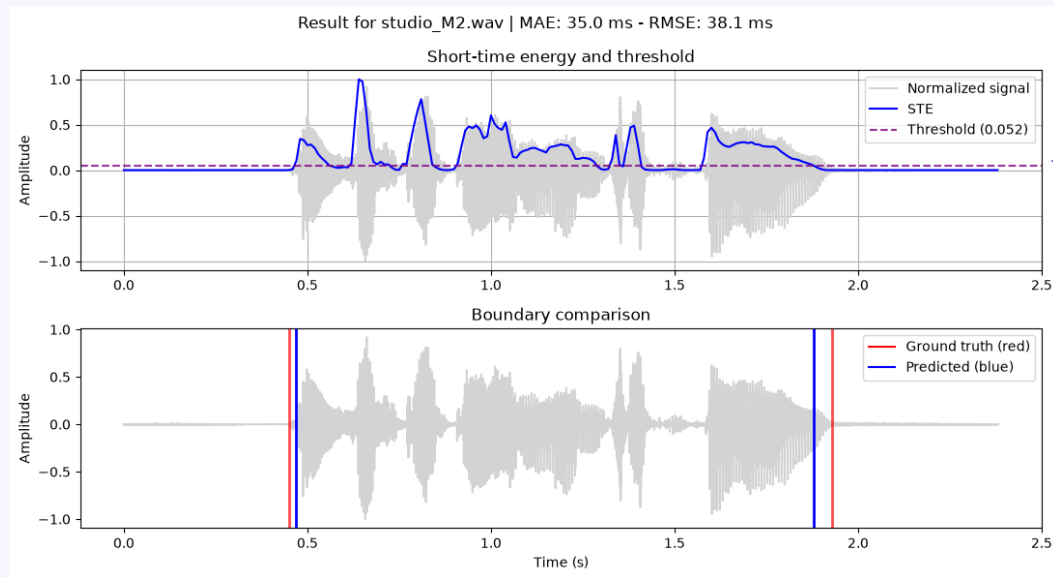
02 SIMPLE STATISTICS: IDEA



02 SIMPLE STATISTICS: IDEA



02 SIMPLE STATISTICS: IDEA



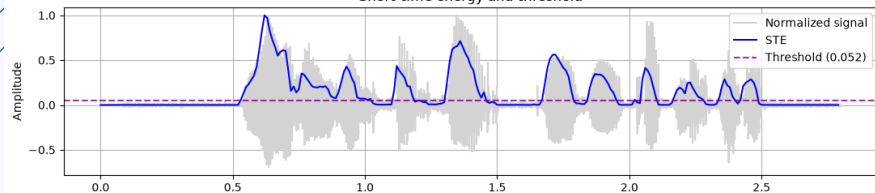
Repeat the algorithm for each Testing signal. After computing the value STE, for each frame:

- **If $STE > \text{Threshold}$ $\Rightarrow$ Speech (Including silence $< 200\text{ms}$).**
- **Otherwise $\Rightarrow$ Silence.**

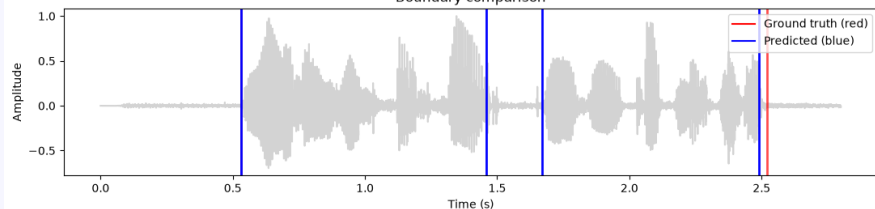
02 SIMPLE STATISTICS: RESULTS

Result for phone_M2.wav | MAE: 15.0 ms - RMSE: 21.2 ms

Short-time energy and threshold



Boundary comparison

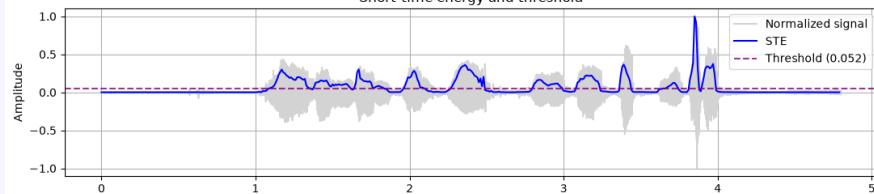


MAE **15ms** | RMSE **21.21ms**

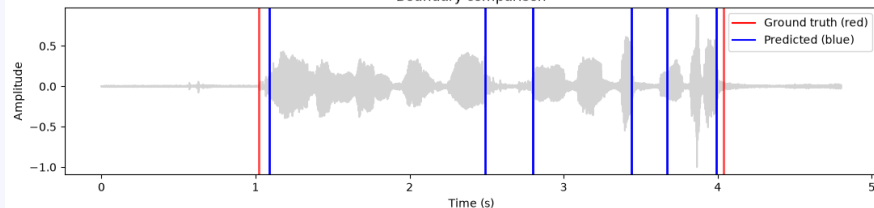
- **Accuracy:** Highly accurate, very few errors.
- **Reason:** STE effectively captures low-frequency male voice energy. Error is due to frame shift.

Result for phone_F2.wav | MAE: 60.0 ms - RMSE: 60.8 ms

Short-time energy and threshold



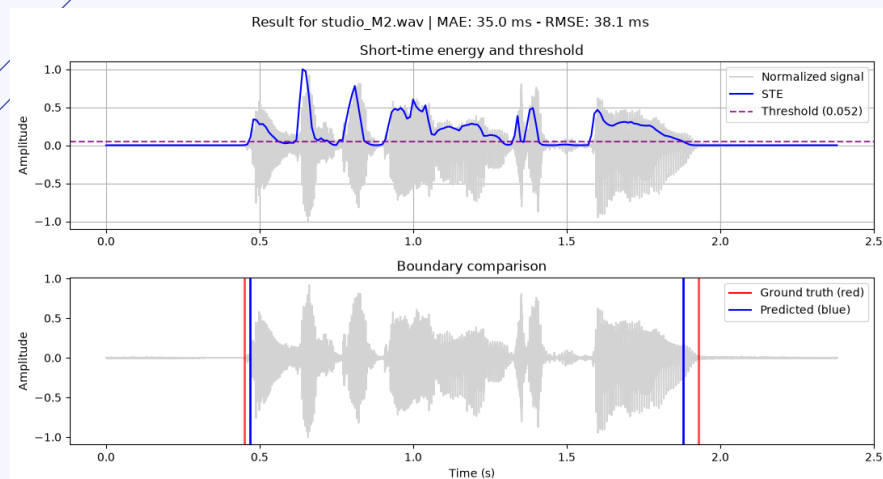
Boundary comparison



MAE **60ms** | RMSE **60.83ms**

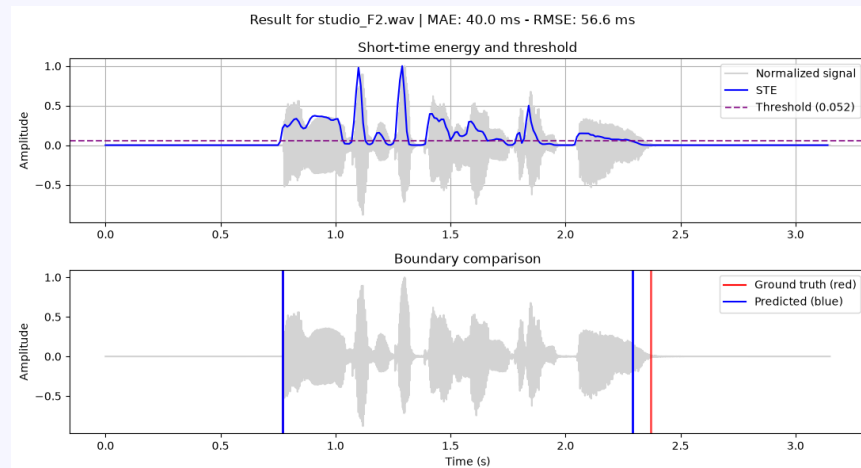
- **Accuracy:** Boundary deviation and shifting at speech edges.
- **Reason:** Female voices contain unvoiced sounds (low STE) at word edges, causing early or late cuts.

02 SIMPLE STATISTICS: RESULTS



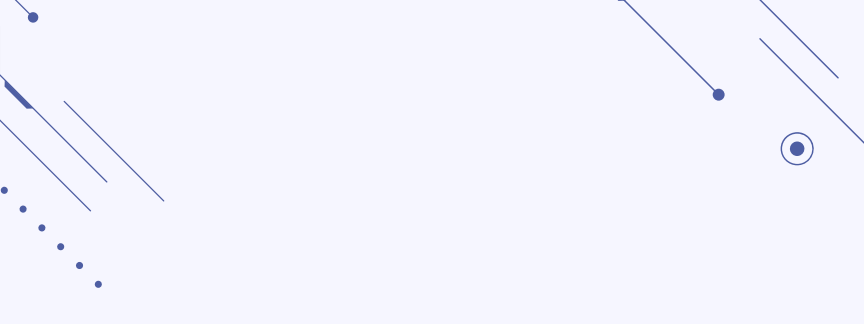
MAE **35ms** | RMSE **38.08ms**

- **Accuracy:** Fairly accurate; slight delay compared to ground truth.
- **Reason:** STE works well for male voices in studio; error from smoothing/post-processing.



MAE **40ms** | RMSE **56.57ms**

- **Accuracy:** Relatively shifted at speech edges.
- **Reason:** Weak unvoiced sounds make STE less sensitive, causing phase delay from smoothing.

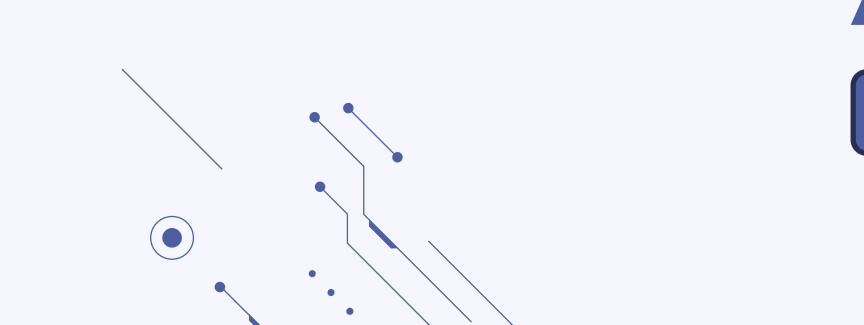


03



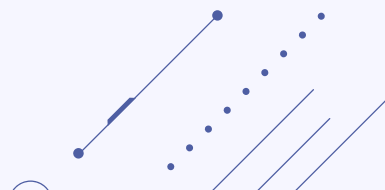
DUAL-THRESHOLD ENDPOINT DETECTION ALGORITHM

Presented by Huỳnh Hiếu Nghĩa



03 DUAL-THRESHOLD: IDEA

Start



03 DUAL-THRESHOLD: IDEA

Start

Audio Signals
(.wav and .lab files)

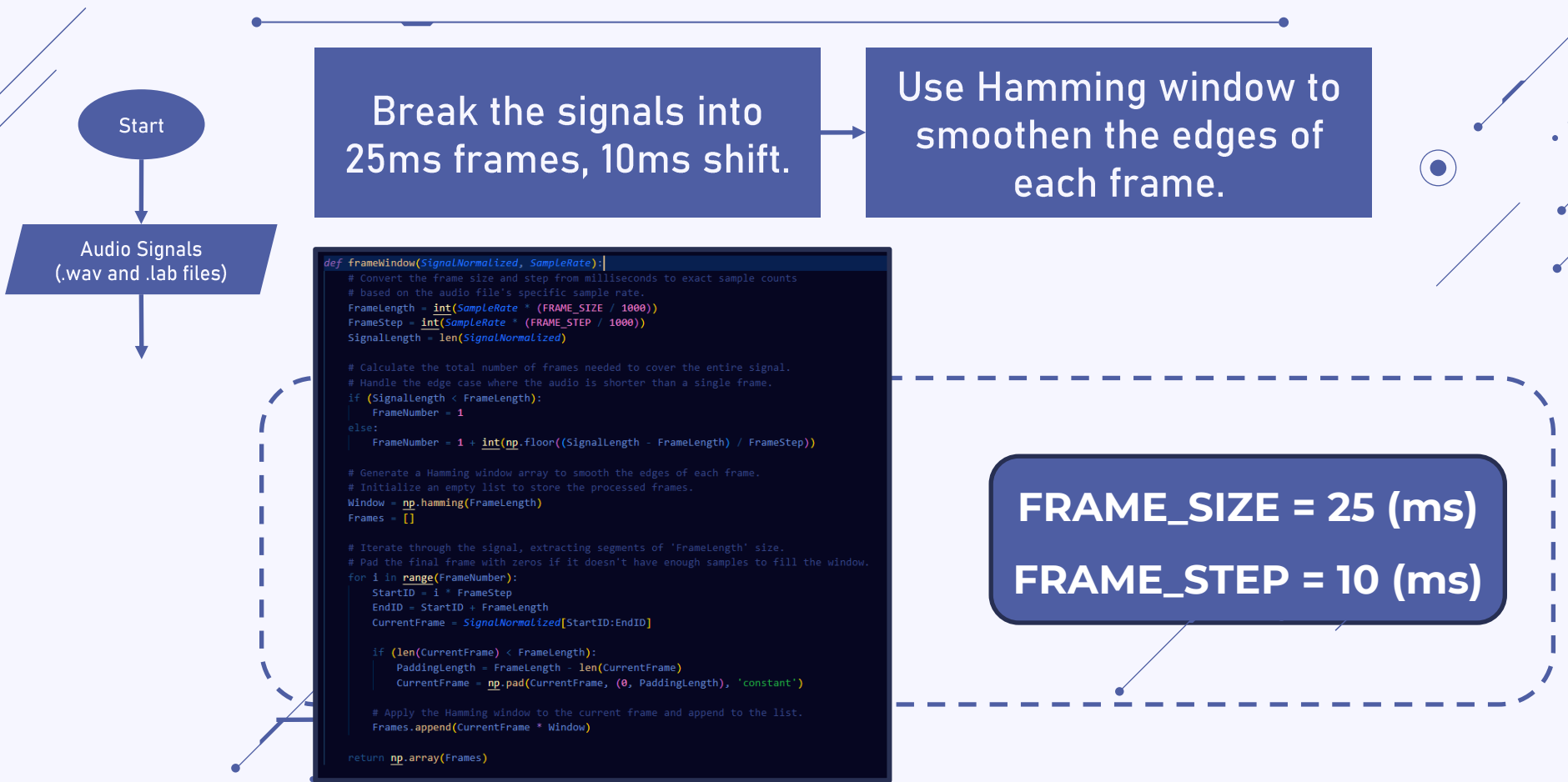
▼ TinHieuHuanLuyen

- phone_F1.lab
- phone_F1.wav
- phone_M1.lab
- phone_M1.wav
- README.txt
- studio_F1.lab
- studio_F1.wav
- studio_M1.lab
- studio_M1.wav

Read files from
TinhieuHuanLuyen directory.

```
def readAudio(path):  
    # Open the wave file, extract basic properties like sample rate and frame count,  
    # and read the raw byte data into a buffer.  
    with wave.open(path, 'rb') as WavFile:  
        SampleRate = WavFile.getframerate()  
        SampleAmount = WavFile.getnframes()  
        RawData = WavFile.readframes(SampleAmount)  
  
    # Convert raw bytes to a 16-bit integer array.  
    # Normalize the signal by dividing by the maximum absolute amplitude  
    # to ensure all values fit within the [-1.0, 1.0] range.  
    Signal = np.frombuffer(RawData, dtype=np.int16)  
    MaxAmp = np.max(np.abs(Signal))  
    if MaxAmp > 0:  
        SignalNormalized = Signal / MaxAmp  
    else:  
        SignalNormalized = Signal  
  
    # Generate a linear time axis array that maps each sample to its exact  
    # timestamp in seconds for later plotting purposes.  
    Duration = SampleAmount / SampleRate  
    TimeAxis = np.linspace(0, Duration, num=SampleAmount)  
    return SignalNormalized, SampleRate, TimeAxis
```

03 DUAL-THRESHOLD: IDEA



03 DUAL-THRESHOLD: IDEA

Calculate value STE, MA and ZCR of each frame.

```
def calculate_STE(Frames):  
    return np.sum(Frames ** 2, axis=1)  
  
def calculate_MA(Frames):  
    return np.sum(np.abs(Frames), axis=1)  
  
def calculate_ZCR(Frames):  
    Signs = np.sign(Frames)  
    SignChanges = np.abs(np.diff(Signs, axis=1)) > 0  
    return np.sum(SignChanges, axis=1)
```

For w-th frame:

$$STE = \sum_{n=1}^N x_w^2(n)$$

$$MA = \sum_{n=1}^N |x_i(n)|$$

$$ZCR = \frac{1}{2N} \sum_{n=1}^N |\text{sign}(x_w[n]) - \text{sign}(x_w[n-1])|$$

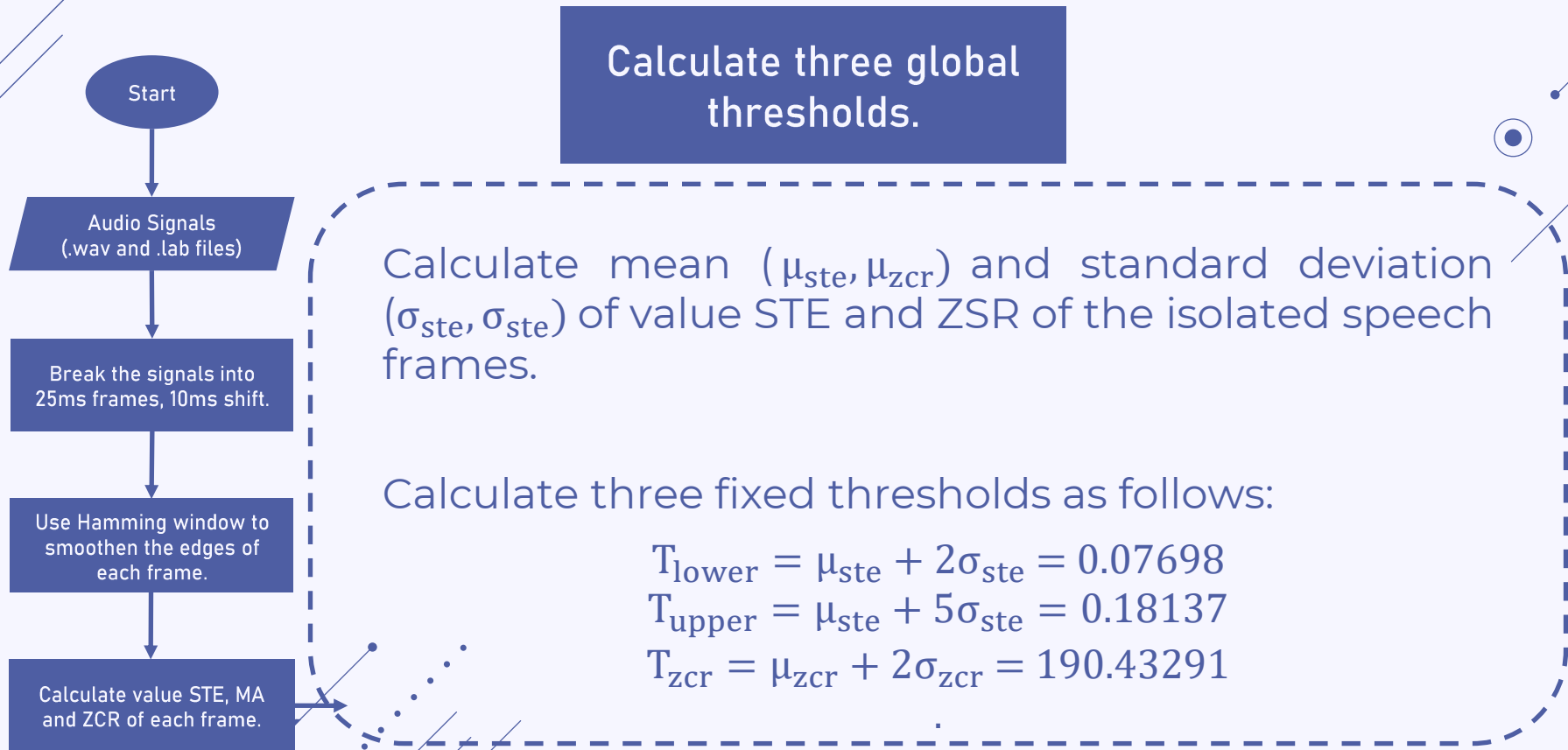
Start

Audio Signals
(.wav and .lab files)

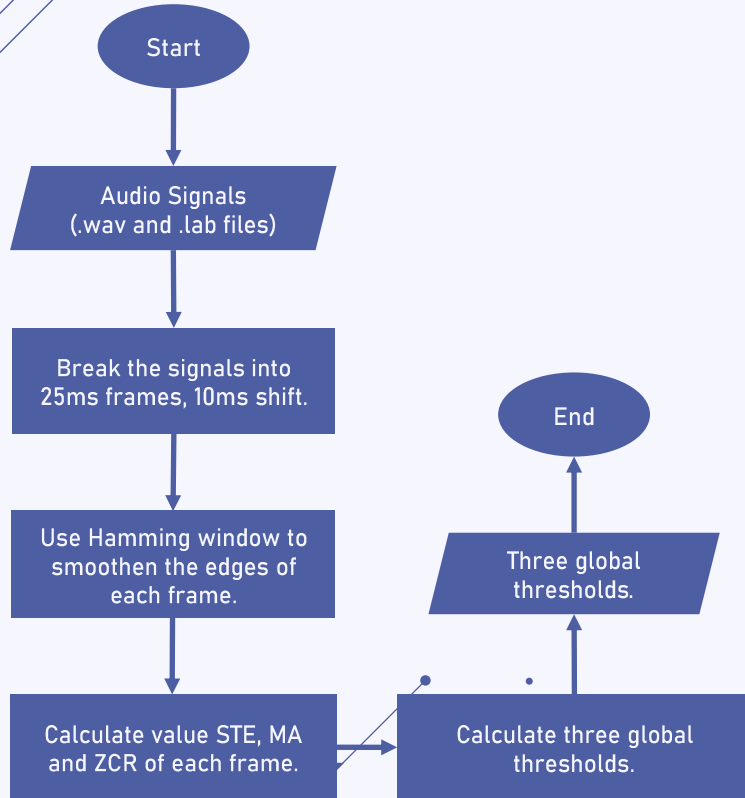
Break the signals into
25ms frames, 10ms shift.

Use Hamming window to
smoothen the edges of
each frame.

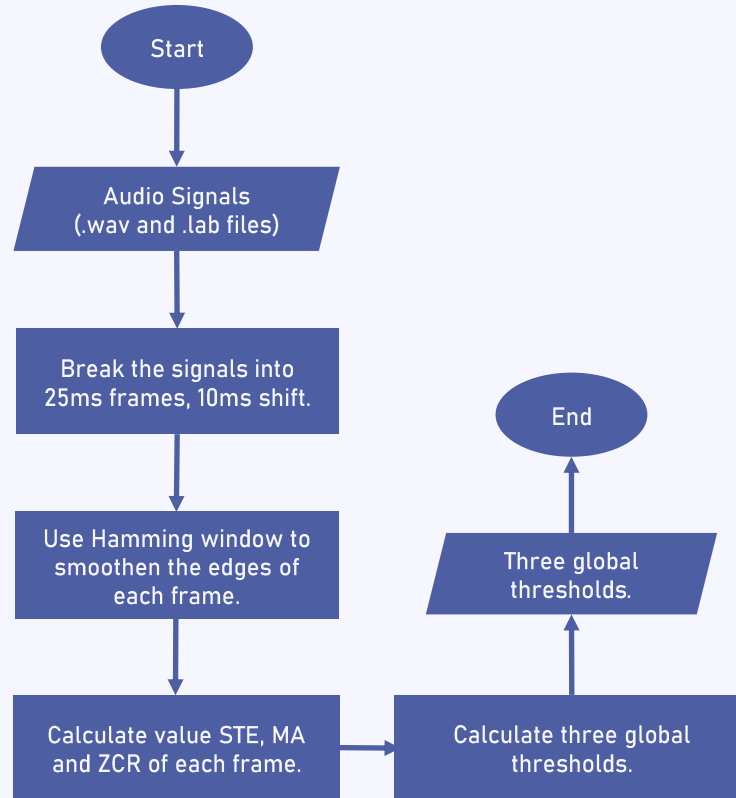
03 DUAL-THRESHOLD: IDEA



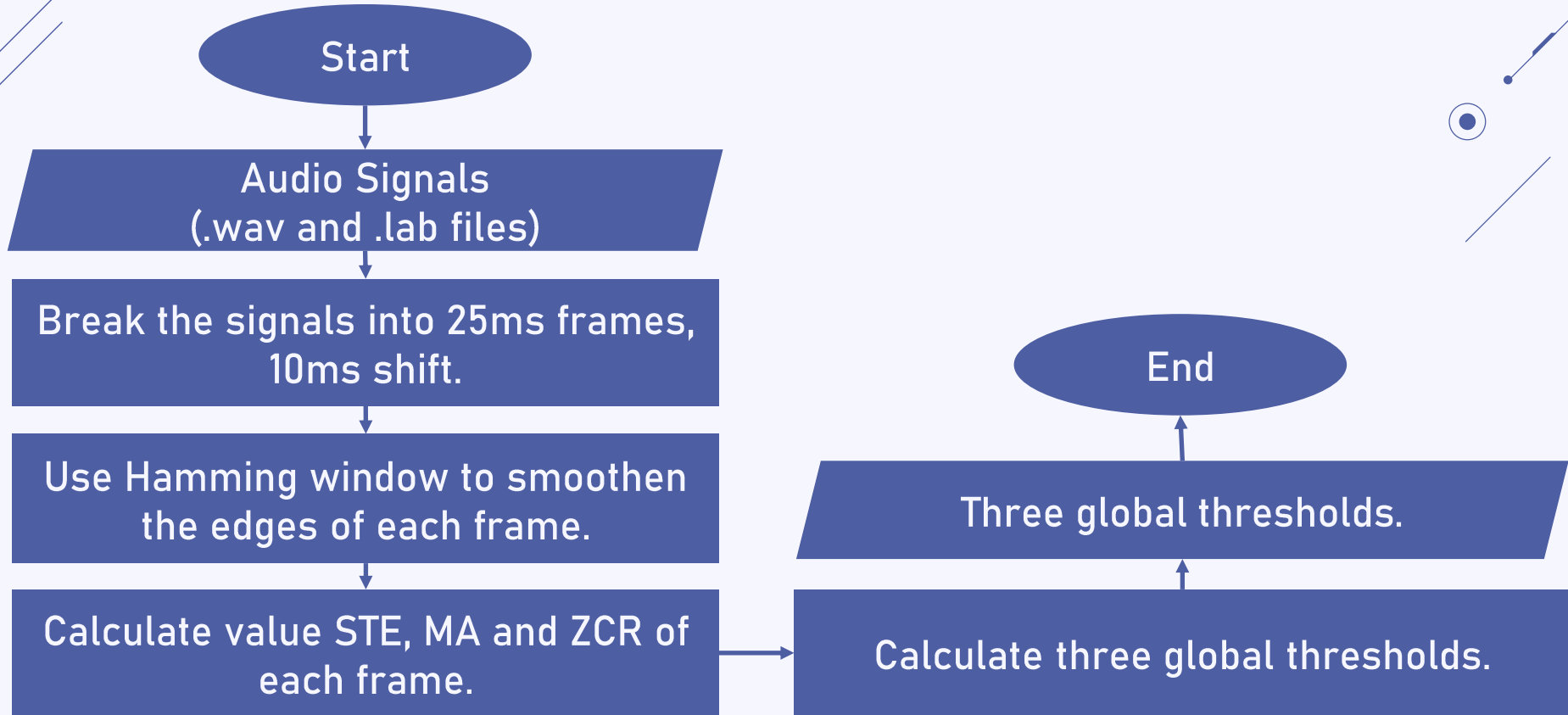
03 DUAL-THRESHOLD: IDEA



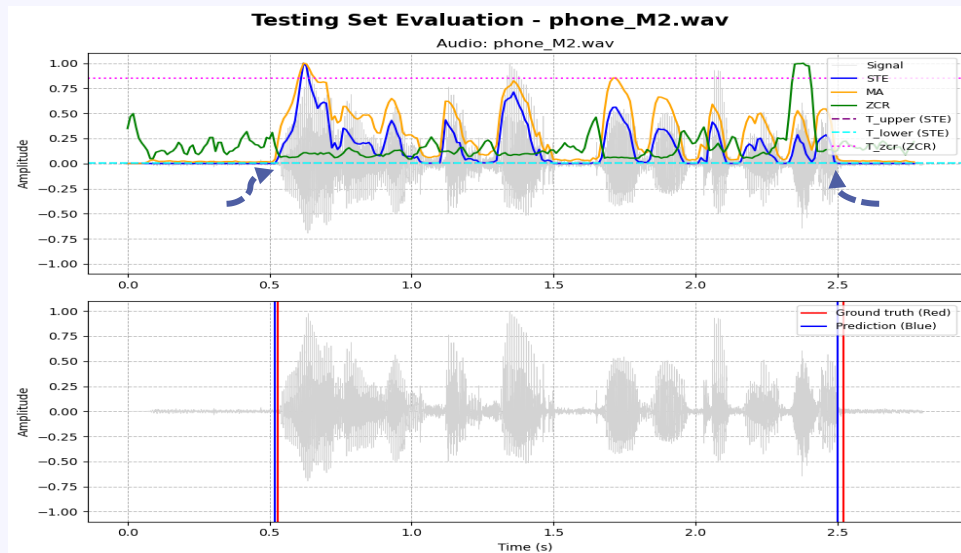
03 DUAL-THRESHOLD: IDEA



03 DUAL-THRESHOLD: IDEA



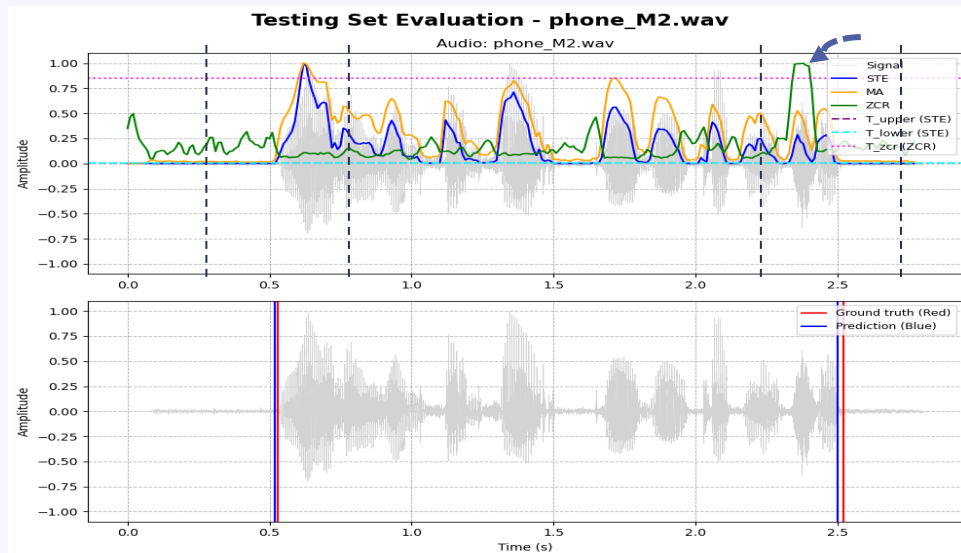
03 DUAL-THRESHOLD: IDEA



Repeat the algorithm for each Testing signal. After computing the STE/MA and ZCR values:

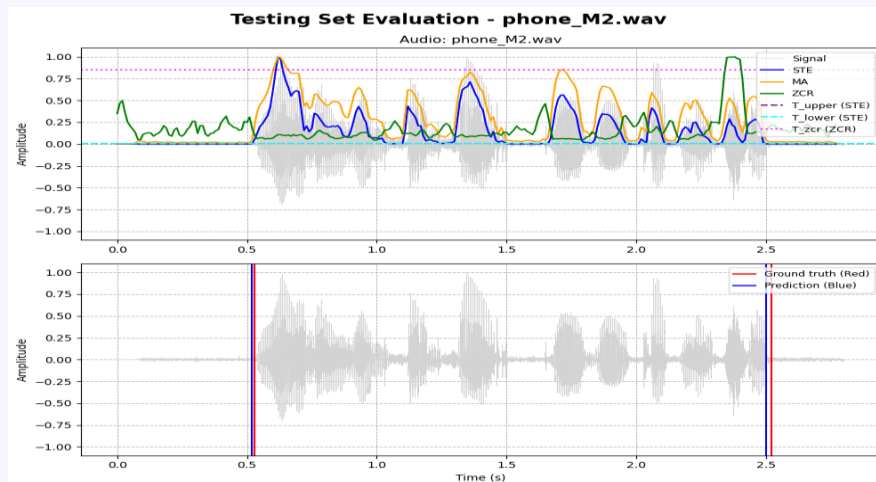
- **STE Segmentation:** Trigger speech flag when **STE** > T_{upper} , setting boundaries where STE drops **below** T_{lower} .

03 DUAL-THRESHOLD: IDEA



- **ZCR Refinement:** Scan 250 more ms (25 more frames) outward of said boundaries, expand them if $ZCR > T_{zcr}$ to include unvoiced consonants (**Including silence < 200ms**).

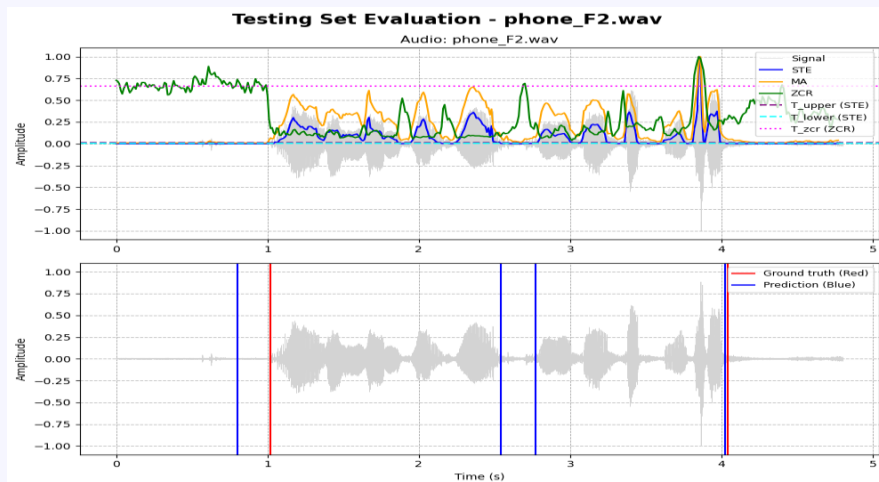
03 DUAL-THRESHOLD: RESULTS



MAE **15ms** | RMSE **15.81ms**

- **Accuracy:** Generally accurate, the blue prediction boundaries closely align with the red ground truth lines throughout the entire file.
- **Reason:** The volume and noise floor of this specific file closely match the average statistics computed during the training phase, allowing the static thresholds to act effectively.

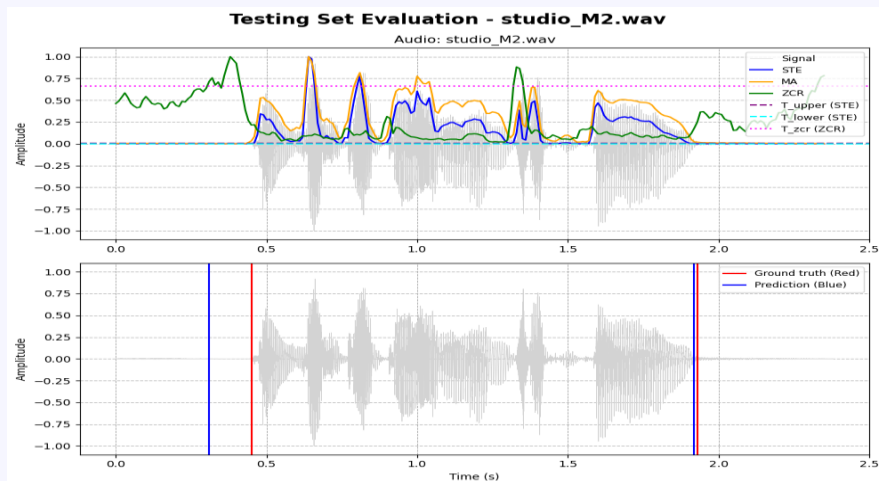
03 DUAL-THRESHOLD: RESULTS



MAE **860ms** | RMSE **1072.01ms**

- **Accuracy:** Highly inaccurate (Severe False Negatives). The blue prediction lines completely miss the speech segments occurring in the second half of the audio.
- **Reason:** Speaker's volume is too low; the STE curve never crosses the T_{upper} threshold, failing to trigger the state machine.

03 DUAL-THRESHOLD: RESULTS



MAE **75ms** | RMSE **99.25ms**

- **Accuracy:** Slightly inaccurate (Boundary stretching), the blue boundaries are wider than the red ground truth lines.
- **Reason:** High background hiss elevates the Zero-Crossing Rate (ZCR), causing the algorithm to mistake noise for unvoiced consonants and falsely stretch boundaries past the T_{zcr} threshold.

03 DUAL-THRESHOLD: RESULTS



MAE **20ms** | RMSE **20ms**

- **Accuracy:** Slightly inaccurate (False Positives). The blue prediction boundaries trigger and end slightly early.
- **Reason:** The resting background noise floor in this recording is louder than the training average, causing standard ambient noise to frequently cross the lower cyan T_{lower} threshold and stretch the boundaries outward.

The background features decorative geometric patterns in the corners, consisting of thin blue lines, dots, and circles. In the top-left, there are several parallel lines and a small cluster of dots. In the top-right, a circle with a dot inside is visible. In the bottom-left, there are more parallel lines and a circle with a dot. In the bottom-right, there is a complex arrangement of lines, dots, and a circle with a dot.

04

**COMPARISON
TABLE**

04

COMPARISON TABLE

Algorithms	phone_M2 .wav	phone_F2 .wav	studio_M2 .wav	studio_F2 .wav	Average
Histogram	MAE: 10ms RMSE: 10.31ms	MAE: 20ms RMSE: 21.36ms	MAE: 10ms RMSE: 10.31ms	MAE: 17.51ms RMSE: 18.21ms	MAE: 14.38ms RMSE: 15.05ms
Simple Statistics	MAE: 15ms RMSE: 21.21ms	MAE: 60ms RMSE: 60.83ms	MAE: 35ms RMSE: 38.08ms	MAE: 40ms RMSE: 56.57s	MAE: 37.5ms RMSE: 44.17ms
Dual-Threshold Endpoint Detection	MAE: 15ms RMSE: 15.81ms	MAE: 860ms RMSE: 1072.01ms	MAE: 75ms RMSE: 99.25ms	MAE: 20ms RMSE: 20ms	MAE: 242.5ms RMSE: 301.77ms

The background features abstract geometric patterns in the corners, consisting of thin blue lines, dots, and circles, resembling a circuit board or a stylized architectural design.

**THANK YOU
FOR WATCHING!**
